

Výrazy a příkazy

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická v Jihlavě

© Karel Richta, 2023

Základy strukturovaného programování, ZSP 10/2023, Lekce 3

<https://moodle.vspj.cz/course/view.php?id=202851>



Výrazy

- Výrazy v programu popisují požadované operace s daty – chceme data sčítat, násobit apod.
- C má bohatý repertoár **operací**: aritmetické, bitové, logické, relační, operace s ukazateli
- Výraz může označovat **modifikovatelný objekt** (lvalue) nebo **hodnotu** (rvalue)
- Pořadí vyhodnocení operandů není (až na výjimky) definováno!
- Některé operace mají vedlejší efekt (inkrementace, dekrementace, přiřazení) => v jednom výrazu nad jedním objektem je třeba použít maximálně jeden!

Aritmetické výrazy

- Operace se provádějí v těchto aritmetikách:

`int, unsigned, long, unsigned long,`
`float, double, long double`

- Před provedením operace může dojít k *implicitní konverzi operandů*:

- `char, short (signed, unsigned)` a `enum` jsou před provedením operace konvertovány na `int` (nebo `unsigned int`) - tzv. *roztahení (integral promotion)*
- jméno pole prvků typu T je konvertováno na typ ukazatel na T
- jméno funkce je konvertováno na typ ukazatel na funkci

Aritmetické výrazy (pokračování)

U většiny operací s aritmetickými operandy se provádějí tzv. ***běžné aritmetické konverze*** (usual arithmetic conversions):

- operandy se konvertují pomocí roztažení, a následně se převedou na stejný typ, dle tabulky

operand typu ...zůstane	druhý se převede na ...
<code>long double</code>	<code>long double</code>
<code>double</code>	<code>double</code>
<code>float</code>	<code>float</code>
<code>unsigned long</code>	<code>unsigned long</code>
<code>long</code>	<code>long</code>
<code>unsigned</code>	<code>unsigned</code>
<code>int</code>	<code>int</code>

Typy číselných literálů

- Typy číselných literálů:

12 int 12L long

3.4 double 3.4F float 3.4L long double

- Příklady:

'a' + 'b' konverze na int, výsledek int

1 / 2 celočíselné dělení, výsledek 0

1.0 / 2.0 reálné dělení, výsledek double

1 / 2.0L konverze na long double

Aritmetické operace

- Operandy aritmetického typu, provádějí se běžné aritm. konverze
- Unární: $+$ $-$
- Binární: $+$ $-$ $*$ $/$ $\%$
- Dělení je podle typu operandů buď celočíselné nebo v pohyblivé řádové čárce
- Operace $\%$ je zbytek po dělení, musí mít operandy celočíselné, a je definována takto:
$$x == (x / y) * y + x \% y$$
- Operace $+$ a $-$ jsou definovány též pro ukazatele (viz dále)

Relační operace

- Operandy aritmetického typu, provádějí se běžné aritm. konverze.
- Výsledek je vždy typu **int** (v C, v C++ je typu **bool**):
 - 0 ... relace neplatí
 - 1 ... relace platí (přesněji, jakákoliv nenulová hodnota se chápe jako pravdivá)
- Binární relační operátory:
< > <= >= == !=
- Relační operátory jsou definovány též pro ukazatele

Logické operace

- Operandy skalárního typu
- Výsledek v C je typu **int** (0 nebo 1)
- Pozn.: Výsledek v C++ je typu **bool** (false nebo true)
- Operandy se vyhodnocují zleva do prava, druhý operand se nevyhodnocuje, je-li výsledek dán prvním operandem

!	logická negace - výsledek 1, má-li operand hodnotu 0, jinak je výsledek 0
&&	logický součin - výsledek 1, jsou-li oba operandy nenulové, jinak 0
	logický součet - výsledek 1, je-li alespoň jeden operand nenulový, jinak 0

Příklad: načtení řádky znaků

```
#define MAXDELKA 80
```

```
char radek[MAXDELKA+1];
```

```
int i = 0;
```

```
int c;
```

```
int konec = 0;
```

```
while ( !konec ) {
```

```
    if ( (i <= MAXDELKA) &&
```

```
        ((c = getchar()) != '\n') &&
```

```
        (c != EOF) )
```

```
    {
```

```
        radek[i] = c;
```

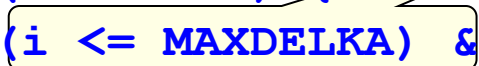
```
        i = i + 1;
```

```
    }
```

```
    else konec = 1;
```

```
}
```

když není místo,
vůbec se nečte



Inkrementace a dekrementace

- Unární operace, operand skalárního typu, modifikovatelný, označení místa v paměti
- Lze zapsat prefixově nebo postfixově
- Má vedlejší efekt: změni hodnotu operandu
- Inkrementace: **++** Dekrementace: **--**
- Prefix: **++X** **--X**
 inkrementuje (dekrementuje) X, hodnotou je změněné X
- Postfix: **X++** **X--**
 inkrementuje (dekrementuje) X, hodnotou je X před změnou

Příklady:	před	operace	po
	x == 1	y = ++x	x == 2, y == 2
	x == 1	y = x++	x == 2, y == 1
	x == 1	y = --x	x == 0, y == 0
	x == 1	y = x--	x == 0, y == 1

Přiřazení

- Syntaxe: **$X = Y$**
- Levý operand musí být modifikovatelný, označení místa v paměti
- Přiřazení má hodnotu a vedlejší efekt
- Hodnotou výrazu je přiřazovaná hodnota, typem je typ levé strany
- Pozor: plete se $=$ a $==$!
- Přípustné typy operandů:
 - levá a pravá strana jsou téhož skalárního typu nebo téhož typu **struct** nebo **union** (pro pole není přiřazení dovoleno)
 - jsou-li typy levé a pravé strany skalární, avšak různé, musí být hodnota pravé strany konvertibilní na typ levé strany
- Složené přiřazení:
 $+=$ $-=$ $*=$ $/=$ $\%=$ $\&=$ $|=$ $\wedge=$ $\ll=$ $\gg=$
- Význam:
 - $X \text{ op} = Y$ je zkratkou za $X = X \text{ op } Y$

Konverze při přiřazení

Následující tabulka udává možné konverze při přiřazení

typ pravé strany	typ levé strany	poznámka ke konverzi
racionální	kratší racionální	zaokrouhlení mantisy
racionální	delší racionální	doplnění mantisy nulami
racionální	celočíselný	odseknutí necelé části
celočíselný	racionální	možná ztráta přesnosti
celočíselný	kratší celočíselný	odseknutí vyšších bitů
celočíselný unsgn.	delší celočíselný	doplnění nulových bitů
celočíselný sgn.	delší celočíselný	rozšíření znaménka
0	T *	
T * nebo 0	void *	

Reference a dereference

- Reference (adresa):

& X kde X je označení datového objektu nebo funkce

- Je-li X typu T , pak $\&X$ je typu T^* , tzn. ukazatel na T
- Dereference (zpřístupnění objektu, na který ukazuje ukazatel):

*** X** kde X je výraz typu ukazatel

- Je-li X typu T^* , pak $*X$ je typu T
- Příklady:

<code>int i, *pi = &i;</code>	<code>/* pi obsahuje adresu i */</code>
<code>char c, *pc = &c;</code>	<code>/* pc obsahuje adresu c */</code>
<code>*pi = 25;</code>	<code>/* do i se uloží 25 */</code>
<code>*pc = 'a';</code>	<code>/* do c se uloží 'a' */</code>
<code>pi = pc;</code>	<code>/* chyba */</code>

Reference a dereference (pokračování)

- Příklady:

```
char str[] = "Hello, word";  
char *ptr = &str[2];  
/* ptr obsahuje adresu 3. prvku řetězu */  
*ptr = '1';  
/* změní se 3. prvek řetězu */
```

Inicializace

- Datové objekty (proměnné, konstanty) se inicializují v době vzniku
 - statické na začátku programu
 - ostatní na začátku funkce (bloku)
- Implicitní inicializace
 - statické objekty jsou vynulovány
 - ostatní nemají definovanou hodnotu
- Explicitní inicializace
 - *deklarátor* = výraz nebo
 - *deklarátor* = { seznam výrazů }
- Statické objekty lze inicializovat pouze konstantními výrazy
- Příklady:
float A = 10;
register int B = A*A;

Příklady na inicializaci polí

```
int P[3] = {1, 2, 3};  
/* inicializací lze zadat počet prvků pole */  
int Q[] = {1, 2, 3};  
int PP[4][3] = {{1, 3, 5}, {2, 4, 6}, {3, 5, 7}};  
/* následující inicializace má stejný efekt */  
int PP[4][3] = {1, 3, 5, 2, 4, 6, 3, 5, 7};  
/* inicializace nemusí být úplná */  
int QQ[][3] = { {1}, {2}, {3}, {4} };  
/* pole má 4 řádky, inicializován je jen 1. sloupec  
   ostatní jsou vynulovány */  
char Msg[] = "text";  
char Msg[5] = {'t', 'e', 'x', 't', '\0'}; /* totéž */  
char Mag[4] = "text"; /* bez závěrečné 0 */
```

Příklad: Inicializace tabulky

```
#define MAXZAM 10

typedef struct osoba {
    unsigned int ID;
    char jmeno[25];
    char prijmeni[36];
    float plat;
} ZAM;                /* definice typu ZAM */

ZAM tab[MAXZAM] = {
    { 1, "Karel", "Novák", 10000.0 },
    { 2, "Jana", "Nováková", 8000.0 },
    { 3, "Dáša", "Poláková", 14000.0 },
};                    /* tabulka zaměstnanců */
int pocet = 3;        /* počet zaměstnanců */
```

Příkazy

- Obecně:
 - Příkazy se dělí na jednoduché a strukturované.
 - Všechny jednoduché příkazy jsou zakončeny středníkem.
 - Podmínka v cyklech a podmíněném příkazu se uvádí jako výraz skalárního typu uzavřený do kulatých závorek; je splněna, je-li hodnota výrazu různá od nuly (resp. nabývá hodnoty true v C++).

- Výrazový příkaz:

výraz ;

- smysl mají jen výrazy s vedlejším efektem

```
x = y + 2; y++; f(a, 2); g();  
g; /* příkaz bez vedlejšího efektu */
```

- Prázdný příkaz:

;

- lze použít např. jako prázdné tělo cyklu:

```
for (i = 0; i < 10; a[i++] = 0);
```

Příkaz bloku

- Příkaz bloku:

{ posloupnost deklarací posloupnost příkazů } - pozn. v C++ se mohou míchat

- deklarace jsou v bloku lokální,
- v bloku platí též deklarace z nadřazeného kontextu,
- lokální deklarace zastíní deklaraci stejného identifikátoru z nadřazeného kontextu,
- deklarace mohou chybět (složený příkaz).

```
{  
    int a, b;  
    a = 10;  
    { int c = a; /* c = 10 */  
        int a = 20;  
            c = a;      /* c = 20 */  
            a = 0;  
        }  
    printf("%d", a);    /* vypíše se 10 */  
}
```

Podmíněný příkaz

- Podmíněný příkaz:

if (*podmínka*) *příkaz*

if (*podmínka*) *příkaz1* **else** *příkaz2*

- else** se vztahuje k nejbližšímu předcházejícímu **if** ve stejném bloku (závorkuje se zprava)

```
if (a > b) m = a; else m = b;
```

```
if (a > b)  
    if (a > c) m = a; else m = c;
```

```
if (a > b) {  
    if (a > c) m = a;  
} else m = b;
```

Příkazy cyklu

- Příkaz **while**:

while (*podmínka*) *příkaz*

- tělo cyklu se provádí, dokud je splněna podmínka
- podmínka se vyhodnocuje na začátku

- Příkaz **do**:

do *příkaz* **while** (*podmínka*) ;

- tělo cyklu se provádí, dokud je splněna podmínka
- podmínka se vyhodnocuje na konci těla
- jediný strukturovaný příkaz zakončený středníkem

- Příkaz **for**:

for (*výraz1* ; *výraz2* ; *výraz3*) *příkaz*

- příkaz má stejný efekt, jako příkazy:

výraz1; while (výraz2) { příkaz ; výraz3 ; }

- V C++11 existují další tvary cyklů pro kontejnery:

for (*deklarace : rozsah*) *příkaz*

Příklad – cyklus while

// Cyklus, kdy počet průchodů není předem znám, může být i žádný

```
#include <stdio.h>
```

```
int main() {  
    char radek[100] = "Hello!";  
    int i = 0;  
    int pocet = 0;  
    while (radek[i++]) pocet++;  
    printf("Delka retezce je: %d\n", pocet);  
}
```

Příklad – cyklus do while

// Cyklus, kdy počet průchodů není předem znám, ale projde alespoň jednou

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str[100];
```

```
    do {
```

```
        printf("Vloz text: ");
```

```
        scanf_s("%s", str, 100);
```

```
        printf("Vlozil jsi: %s\n", str);
```

```
    } while (strcmp(str, "goodbye"));
```

```
    system("pause");
```

```
    return 0;
```

```
}
```

Příklad - cyklus for

```
// Cyklus for řízený ručně
#include <stdio.h>
#include <stdlib.h>

int main() {
    int A[10], soucet = 0;
    for (int i = 0; i < 10; i++) {
        A[i] = i;
    }
    for (int j = 9; j >= 0; j--) {
        soucet += A[j];
    }
    printf("Soucet je: %d\n", soucet);
}
```

Příklad - cyklus **for** řízený rozsahem

```
// Cyklus for řízený rozsahem (range-based)
#include <stdio.h>
#include <stdlib.h>

int main() {
    char str[7];
    strcpy(str, "Hello!" );
    for (char c : str) {
        printf("[%c]",c);
    }
    printf("\n");
}

// typ proměnné v deklaraci lze nechat odvodit
for (auto c : str)
```

The End